

Student Attrition Machine Learning: Spec-Driven Documentation

This document accumulates all Spec-Driven Development methodology documentation for the Sprint 2 student-attrition machine-learning workflow. The specification is the source of truth from which the technical plan, implementation tasks, code, tests, and supporting documentation are derived.

Document set

SDD phase	File	Purpose
0 - Constitution	<code>constitution.md</code>	Persistent project principles, technology stack, ethical rules, and guardrails that constrain every design and implementation decision.
1 - Specify	<code>specification.md</code>	Sprint 2 machine-learning user stories, acceptance criteria, priorities, story points, target behaviour, constraints, and out-of-scope work. Defines the what and why.
2 - Clarify	Integrated into <code>specification.md</code> and <code>plan.md</code>	Records the confirmed decisions made before planning, including the student-level target, Spark ML implementation, recall as the primary metric, the 0.50 threshold, and the exclusion of model-comparison experiments.
3 - Plan	<code>plan.md</code>	Technical blueprint for preparing the student-level dataset, preprocessing features, training and evaluating the Random Forest, logging the run in MLflow, and writing predictions to Delta Lake.
4 - Tasks	<code>tasks.md</code>	Atomic, testable, dependency-ordered implementation tasks derived from the approved specification and plan.

The PDF files supplied with this package are human-readable exports of the same approved Markdown documentation. The Markdown files are the controlling files supplied to the AI coding agent.

Current status

- **Sprint:** Sprint 2.
- **Documentation status:** Finalised following human review and approved for implementation.
- **Implementation status:** Complete. Approved by human review on 2026-08-06 after successful execution in Databricks. All tasks T1 to T32 are marked complete in `tasks.md`.
- **Output table:** `workspace.student_aggregate.student_attrition_risk_prediction` — one row per `student_deidentified_hash`, written by `student_attrition_machine_learning.py`.
- **Known limitations:** L-1 to L-3 in `plan.md` govern how stored risk percentages should be read.

Confirmed project scope

- The model predicts twelve-month student attrition: whether an individual student exits university education, not whether the student leaves a particular course.
- The supervised target is `is_twelve_month_student_attrition`; `is_twelve_month_course_attrition` is excluded from predictive features to prevent target leakage.
- The implementation uses a Spark ML Random Forest, a reproducible 70/15/15 train-validation-test split, recall as the primary evaluation metric, and a fixed 0.50 risk threshold.
- The workflow must prepare the modelling data, train and evaluate the model, generate an attrition-risk percentage and Boolean risk flag for every applicable student, and persist the validated results in a created queryable Delta table.
- The prediction Delta table is linked by `student_deidentified_hash` and contains the information required for the Databricks application to retrieve each student's attrition-risk result, including the percentage, Boolean flag, prediction threshold, MLflow run identifier, and scoring timestamp.
- MLflow records model parameters, metrics, and the trained model artifact. Formal model governance, model-comparison experiments, and competing feature-set experiments are outside the Sprint 2 scope.

User-story relationship

The user stories in `specification.md` are numbered independently from the main Product Backlog because they are detailed requirements for the Sprint 2 machine-learning component.

- Machine-learning specification **US-1** is implemented under Product Backlog **US-05: Machine Learning Classification Model**.
- Machine-learning specification **US-2** is implemented under Product Backlog **US-06: Student Attrition Risk Percentage Generation**.
- Machine-learning specification **US-3** is implemented under Product Backlog **US-07: Student Attrition Risk Prediction Delta Table**.
- Machine-learning specification **US-4 to US-8** are supporting technical requirements primarily under Product Backlog **US-05**, with relevant support for US-06 and US-07.

These detailed stories may be represented as GitHub sub-issues beneath the corresponding Product Backlog stories rather than added as new top-level Product Backlog items.

Approved implementation workflow

1. Provide `README.md`, `constitution.md`, `specification.md`, `plan.md`, and `tasks.md` to the AI coding agent.
2. The agent reads the controlling SDD files in that order and implements `tasks.md` in dependency order without changing approved behaviour.
3. The agent inspects the synthetic-data generator, existing ML prototype, Delta schemas, table relationships, and available columns as technical implementation sources after reviewing the controlling SDD documents.
4. The agent reports conflicts, unavailable features, leakage risks, technical corrections, known limitations, and proposed refinements.
5. Material refinements that change model behaviour, features, parameters, data grain, outputs, or architecture require human review and approval before they are accepted.
6. A team member executes and tests the implementation in Databricks, reviews it against the acceptance criteria, and authorises completed task and user-story statuses.

Source references

- **Synthetic data generator:** DBLDataGen Synthetic Data Generation V2.
- **Input data:** deidentified synthetic Delta fact and dimension tables in `workspace.student_aggregate`.
 - **Primary target:** `is_twelve_month_student_attrition`.
- **Output table:** `workspace.student_aggregate.student_attrition_risk_prediction`.
- **Enterprise design context:** T115 Student Risk & Intervention Briefing physical solution design.
 - **Parent Product Backlog stories:** US-05, US-06, and US-07 from T115 Product Backlog User Stories.

Constitution: Student Attrition Machine Learning

Persistent principles that constrain the Sprint 2 machine-learning implementation.

1. Project purpose

Develop a machine-learning classification workflow that uses deidentified synthetic student data to estimate whether an individual student will exit university education within the defined twelve-month attrition window. The output supports academic advisors in identifying students who may require additional support.

2. Technology stack

- **Platform:** Databricks using serverless-compatible compute.
- **Processing and modelling:** Python, PySpark, and Spark ML.
- **Storage:** Delta Lake tables in the existing Unity Catalog environment.
- **Experiment tracking:** MLflow for model parameters, metrics, and the trained model artifact.
- **Data source:** deidentified synthetic student fact and dimension tables.

3. Guiding principles

1. **Specification as source of truth.** Required behaviour is defined in the specification before implementation; the plan, tasks, code, and tests remain consistent with it.
2. **Student-level prediction.** The model predicts whether a student exits education, not whether a student leaves one course or whether a course has a high leaving rate.
3. **Single target definition.** The supervised target is `is_twelve_month_student_attrition`.
4. **No target leakage.** The target itself, `is_twelve_month_course_attrition`, and any information that reveals or occurs after the target outcome are excluded from predictive features.
5. **Identifiers are not predictors.** Deidentified student and enrolment hashes are retained for traceability but excluded from the feature vector.
6. **Prediction-time validity.** Every feature must be available at the intended census-date prediction snapshot.
7. **Reproducibility.** Data splitting, preprocessing, training, and evaluation use recorded settings and fixed random seeds where applicable.
8. **Evaluation separation.** Training data fits the model, validation data checks development performance, and test data is reserved for final evaluation.
9. **Privacy by design.** Only deidentified synthetic data is processed; the workflow must not expose or reconstruct real student records.
10. **Human oversight.** AI-generated implementation is reviewed and tested by a team member before acceptance.

4. Data ethics and decision support

- The model output supports academic-advisor judgement and does not automatically determine an intervention.
- Predictions remain linked only to deidentified student identifiers.
- A risk percentage is a model estimate, not a confirmed future outcome.

5. Change management

- Clarify and update the specification before changing required behaviour.
- Derive the technical plan and atomic tasks from the approved specification.
- Do not mark Sprint 2 work complete until acceptance criteria are tested and human-reviewed.

Specification: Student Attrition Machine Learning

Sprint 2 requirements, user stories, acceptance criteria, and clarified behaviour.

This specification defines what the Sprint 2 machine-learning workflow must achieve and why.

Technical implementation details are defined separately in `plan.md` after clarification.

1. Overview and intent

The workflow trains and evaluates a binary classification model using deidentified synthetic student data. It predicts twelve-month student attrition, generates an attrition-risk percentage and risk flag for each applicable student, records the model run through MLflow, and stores the validated results in a queryable Delta table linked by `student_deidentified_hash` so that the Databricks application can retrieve each student's correct prediction result.

2. Stakeholders

Persona	Interest in the model
Academic advisor	Uses risk percentages and flags to identify students who may require support.
Machine-learning engineer	Builds, evaluates, and refines the student-attrition model.
Data analyst	Validates modelling data and retrieves prediction results from Delta tables.
Project team / reviewer	Reviews AI-generated code and confirms Sprint 2 acceptance criteria.
Industry partner / data owner	Requires appropriate use of deidentified synthetic data and a student-level outcome.

3. Story numbering and Product Backlog relationship

The user stories in this specification are numbered independently from the main Product Backlog. They describe detailed requirements for developing the Sprint 2 machine-learning component and do not replace or renumber the project-level Product Backlog stories.

Machine-learning specification story	Parent Product Backlog relationship
US-1	Product Backlog US-05: Machine Learning Classification Model

US-2	Product Backlog US-06: Student Attrition Risk Percentage Generation
US-3	Product Backlog US-07: Student Attrition Risk Prediction Delta Table
US-4 to US-8	Supporting technical requirements primarily under Product Backlog US-05, with relevant support for US-06 and US-07

These detailed machine-learning stories may be implemented as GitHub sub-issues beneath the corresponding parent Product Backlog stories.

4. User stories and acceptance criteria

Acceptance criteria are written in Given / When / Then form.

US-1: Machine Learning Classification Model

As an academic advisor, I want a machine-learning classification model to assess student attrition risk, **so that** students who may require additional support can be identified.

- **Given** the required synthetic student data is available, **when** the classification model is trained and evaluated, **then** it produces student-level attrition-risk predictions.
- **Given** model training is completed, **when** the run is reviewed, **then** the model parameters, evaluation metrics, and trained model artifact are available through MLflow.
- **Given** the model target is configured, **then** the model predicts twelve-month student attrition rather than course attrition.

MoSCoW Priority: Must Have

Story Points: 13

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-2: Student Attrition Risk Percentage Generation

As an academic advisor, I want an attrition-risk percentage generated for each applicable student, **so that** I can compare and prioritise students who may require support.

- **Given** a trained classification model and valid student data are available, **when** inference is executed, **then** a risk percentage from 0 to 100 is generated for every applicable student.
- **Given** a risk percentage has been generated, **when** the approved 0.50 probability threshold is applied, **then** the corresponding Boolean risk flag is produced.
- **Given** inference results are produced, **then** every risk percentage is valid and non-missing.

MoSCoW Priority: Must Have

Story Points: 8

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-3: Student Attrition Risk Prediction Delta Table

As a data analyst, I want student attrition-risk results stored with the correct deidentified student identifiers, **so that** the Databricks application can retrieve each student's correct prediction result.

- **Given** student attrition-risk predictions have been generated, **when** results are persisted, **then** each `student_deidentified_hash` is linked to its risk percentage and risk flag in a queryable Delta table.
- **Given** the prediction table is written, **then** each applicable student appears once in the latest prediction snapshot.
- **Given** the output is validated, **then** risk flags are Boolean and the logical primary key `student_deidentified_hash` contains no duplicates.
- **Given** a stored prediction is retrieved, **then** the table also provides the approved threshold, MLflow run identifier, and scoring timestamp required for traceability.

MoSCoW Priority: Must Have

Story Points: 5

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-4: Student Attrition Target and Leakage Prevention

As a machine-learning engineer, I want target and feature roles defined correctly, **so that** the model learns to predict student attrition without being given outcome information or arbitrary identifiers.

- **Given** the modelling dataset is prepared, **when** the target is selected, **then** `is_twelve_month_student_attrition` is the sole supervised label.
- **Given** the predictive feature set is created, **then** `is_twelve_month_student_attrition` and `is_twelve_month_course_attrition` are excluded from the feature vector.
- **Given** `student_deidentified_hash` and `enrolment_deidentified_hash` are retained for traceability, **then** neither identifier is included as a predictive feature.
- **Given** any additional feature is proposed, **then** it is included only when it is available at the census-date prediction snapshot and does not reveal the outcome.

MoSCoW Priority: Must Have

Story Points: 5

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-5: Modelling-Data Quality

As a machine-learning engineer, I want the modelling dataset validated before training, **so that** model results are based on complete, correctly joined, student-level data.

- **Given** the modelling dataset is built, **when** validation runs, **then** required identifier, target, and feature columns exist with compatible data types.
- **Given** the target is checked, **then** it contains only valid binary values and no missing labels in supervised-modelling rows.
- **Given** modelling grain is checked, **then** each training record represents one deidentified student and duplicate student identifiers are reported or resolved.

- **Given** fact and dimension data are joined, **then** row-count and key checks confirm that joins do not unintentionally duplicate students.
- **Given** model training begins, **then** target class distribution and feature missingness are reported for review.

MoSCoW Priority: Must Have

Story Points: 5

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-6: Training, Validation and Test Split

As a machine-learning engineer, I want separate and reproducible training, validation, and test datasets, **so that** development checks and final evaluation are reliable.

- **Given** the validated modelling dataset is available, **when** it is split, **then** approximately 70 percent is assigned to training, 15 percent to validation, and 15 percent to testing.
- **Given** the split is repeated with unchanged data and settings, **then** the same seed of 42 is used.
- **Given** the workflow is reviewed during development, **then** validation results are inspected before the untouched test dataset is used for final evaluation.
- **Given** the three datasets are checked, **then** no modelling row occurs in more than one split.

MoSCoW Priority: Must Have

Story Points: 3

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-7: Feature Preprocessing

As a machine-learning engineer, I want numeric and categorical features prepared consistently, **so that** the classification model can train and score students without avoidable data errors.

- **Given** numeric features contain missing values, **when** preprocessing information is derived, **then** numeric missing-value handling is based only on training data.
- **Given** categorical features are used, **when** they are prepared for modelling, **then** categories are encoded into a model-compatible representation and unseen inference categories do not cause failure.
- **Given** preprocessing is applied to validation, test, or inference data, **then** the training-derived preprocessing contract is reused and is not recalculated using validation, test, or inference data.
- **Given** identifiers, labels, and leakage columns are present, **then** they are excluded from the assembled feature vector.

MoSCoW Priority: Must Have

Story Points: 5

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

US-8: Model Evaluation

As a project reviewer, I want the classification model evaluated with suitable metrics, **so that** its ability to identify students at risk is transparent before results are used.

- **Given** a trained model and validation data, **when** evaluation is performed, **then** recall is treated as the primary metric and accuracy, precision, F1 score, ROC-AUC, and a confusion matrix are also produced.
- **Given** development validation is complete, **when** final evaluation is performed, **then** the same required metrics are reported on the untouched test dataset.
- **Given** predictions are reviewed, **then** the model produces non-trivial probabilities rather than a constant score or a single predicted class for every student.
- **Given** evaluation results are recorded, **then** they can be traced to the corresponding MLflow run.

MoSCoW Priority: Must Have

Story Points: 5

Planned Sprint: Sprint 2

Status: Accepted (2026-08-06)

5. Clarified Sprint 2 decisions

- **Applicable students:** every valid student row in the prepared modelling dataset is scored.
- **Prediction timing:** the model represents a census-date snapshot and uses only information treated as available by that point.
- **Implementation approach:** Spark ML Random Forest; the full generated dataset is not collected into one pandas DataFrame.
- **Primary metric:** recall; accuracy, precision, F1, ROC-AUC, and confusion matrix remain supporting metrics.
- **Risk threshold:** fixed probability threshold of 0.50.
- **Risk storage:** advisor-facing `attrition_risk_percentage` is stored from 0 to 100.
- **Prediction persistence:** each successful inference run overwrites the latest prediction snapshot.
- **Output table:** `workspace.student_aggregate.student_attrition_risk_prediction`.
- **Model scope:** one approved Spark ML Random Forest and one validated feature set are implemented. Alternative-model benchmarking, competing feature-subset experiments, automated feature selection, broad hyperparameter searches, and research-led model selection are excluded.
- **Permitted refinement:** unavailable, completely NULL, invalid, incompatible, post-outcome, or leakage-prone features may be corrected or removed as part of validation. Such refinements must be reported to the human reviewer.
- **Human approval:** any proposed refinement that changes model behaviour, approved features, parameters, split, threshold, data grain, output schema, or architecture requires human review and approval before it is accepted.
- **Approved feature amendment (2026-08-06):** `course_admission_load_category` and `commencing_continuing_period` were added to the approved feature set following Phase 1 human review, giving 21 predictive features. Field-of-education attributes from the study area and unit dimensions remain excluded so that the course dimension stays the single canonical owner of that concept. Recorded as amendments A-1 and A-2 in `plan.md`.

6. Constraints and non-functional requirements

- Sprint scope: Sprint 2; all stories were accepted following human review on 2026-08-06.
- Synthetic and deidentified student data only.
- Prediction subject: an individual student exiting education, not a course-level leaving rate.
- Primary prediction identifier: `student_deidentified_hash`.
- The prediction Delta table must support retrieval of the correct risk result for each applicable student by the Databricks application.
- Implementation produced by an AI agent must be human-reviewed before acceptance.

7. Out of scope for Sprint 2

- A separate course-attribution model.
- Alternative-model benchmarking, competing feature-set experiments, automated feature selection, broad hyperparameter searches, and research-led model selection.
- Automated intervention decisions or automatic contact with students.
- Generative-AI production of the final academic-advisor briefing.
- Formal MLflow promotion governance, approval roles, aliases, rollback procedures, or release management.
- Automated retraining, drift monitoring, and scheduled production deployment.
- Validation against real student-level records where access has not been supplied by the industry partner.

Plan: Student Attrition Machine Learning

Technical blueprint derived from the clarified Sprint 2 specification.

Objective

Implement one Spark ML Random Forest workflow that prepares student-level modelling data, trains and evaluates a twelve-month student-attrition classifier, generates a risk percentage and Boolean flag for each applicable student, logs the run through MLflow, and writes the latest validated prediction snapshot to a Delta table that the Databricks application can query by `student_deidentified_hash`.

1. Clarified implementation decisions

- Score every valid student row in the prepared modelling dataset.
- Treat the modelling row as a census-date snapshot and use only information available by that point.
- Use Spark ML rather than collecting the complete dataset to pandas.
- Use recall as the primary evaluation metric.
- Use a fixed probability threshold of `0.50`.
- Store risk as a percentage from 0 to 100.
- Overwrite the latest prediction snapshot on each successful inference run.
- Write to `workspace.student_aggregate.student_attrition_risk_prediction`.
- Implement one approved Spark ML Random Forest using one validated feature set.
- Do not conduct alternative-model benchmarking, competing feature-subset experiments, automated feature selection, broad hyperparameter searches, or research-led model selection.
- Removing unavailable, completely NULL, invalid, incompatible, post-outcome, or leakage-prone features is a necessary validation correction rather than a competing feature-set experiment. The agent must report each such correction to the human reviewer.

2. Architecture and data flow

- Read the synthetic student fact table and required dimension tables from `workspace.student_aggregate`.
- Join descriptive course and teaching-period attributes using existing hash keys.
- Create one validated modelling row per `student_deidentified_hash`.
- Separate identifiers, predictive features, the target label, and leakage exclusions.
- Split the modelling dataset into training, validation, and test datasets.
- Derive preprocessing metadata from the training split only and apply it with stateless Spark transformations. Fit `RandomForestClassifier` directly to the prepared training feature vector (see amendment A-6).
- Evaluate the trained model on validation data, then perform final evaluation on test data.
- Log model parameters, metrics, the trained Random Forest artifact, and the training-derived preprocessing contract through MLflow.
- Run inference for all applicable student rows and persist the latest validated prediction snapshot.

- Make the prediction Delta table queryable by the Databricks application using `student_deidentified_hash`.

3. Data source and modelling grain

Primary source fact table:

- `workspace.student_aggregate.rpt_student_management__fact__all_enrolment_eftsl__deidentified`

Supporting dimensions used only where populated attributes are required:

- `workspace.student_aggregate.dwh_curriculum__course`
- `workspace.student_aggregate.dwh_curriculum__course_offering`
- `workspace.student_aggregate.dwh_learning_and_teaching__teaching_period`

Logical modelling grain: one row per `student_deidentified_hash`.

Before modelling, verify that the current synthetic dataset contains one eligible row per student and that joins do not create duplicates.

4. Column roles

Target label:

- `is_twelve_month_student_attrition`, cast to a numeric binary label for Spark ML.

Identifiers retained outside the feature vector:

- `student_deidentified_hash`
- `enrolment_deidentified_hash`

Prohibited leakage columns:

- `is_twelve_month_student_attrition`
- `is_twelve_month_course_attrition`

Initial predictive features, subject to availability and validation:

- `age_at_census`; `socioeconomic_status`; `regional_remote_status`; student gender; international status; First Nations status.
- `attendance_mode`; `eftsl`; `is_commencing` or `commencing_continuing`; enrolment year; teaching period.
- Cumulative credit points enrolled, passed, failed, and withdrawn.
- Course group or course level and populated field-of-education attributes.
- `course_admission_load_category` and `commencing_continuing_period` (added by approved amendment A-1; see section 14).

Raw hash keys are used for joins and traceability only. Unsupported or completely NULL attributes are not included in the feature vector.

Canonical feature ownership. Each feature concept has exactly one owning source. Demographics, student status, EFTSL, cumulative credit points, and commencing status are owned by the fact table; teaching-period and calendar attributes by the teaching-period dimension; course group and field of education by the course dimension. Equivalent attributes appearing in other dimensions are not added merely because their column names can be prefixed.

5. Data validation

- Confirm required source tables and columns exist.
- Confirm `student_deidentified_hash` and the target are non-null for supervised-modelling rows.
- Confirm the target contains only binary values.
- Report target class counts and percentages.
- Report feature missingness and remove unusable all-null features.
- Confirm one row per student after joins and report any duplicate identifiers.
- Confirm no target or leakage column enters the assembled feature vector.
- Record and report any feature removal, join correction, type correction, or other refinement made during preparation.

6. Data split

- Use a deterministic 70/15/15 training, validation, and test split with seed 42, assigned by hashing `student_deidentified_hash` into 100 buckets (see amendment A-4).
- Record split row counts and class distributions.
- Verify that no row appears in more than one split.
- Use validation for development checks only; keep test untouched until final evaluation.

7. Preprocessing

- Numeric features: cast to numeric types. Missing and NaN values are replaced using medians derived exclusively from the training split.
- Categorical features: convert to deterministic manual one-hot columns from training-derived category levels, with one explicit bucket for NULL or previously unseen values.
- Assemble processed columns into a single Spark ML features vector using `VectorAssembler` as a Transformer only. Do not call `fit()` on the assembler.
- Do not apply feature scaling because the selected Random Forest does not require it.
- Derive preprocessing metadata only from training data and reuse that training-derived contract for validation, test, and inference. Do not fit Spark ML preprocessing estimators (`Imputer`, `StringIndexer`, `OneHotEncoder`) or a preprocessing `Pipeline`.

The original design fitted `Imputer`, `StringIndexer`, `OneHotEncoder`, and a Spark ML `Pipeline`. That architecture was replaced by the training-derived, stateless workflow above and is recorded as amendment A-6 (Spark Connect fitted-model size overestimation / [SPARK-57521](#)).

8. Random Forest configuration

- **Classifier:** `pyspark.ml.classification.RandomForestClassifier`
- **Number of trees (RF_NUM_TREES):** 100
- **Maximum depth (RF_MAX_DEPTH):** 7
- **Minimum instances per node (RF_MIN_INSTANCES_PER_NODE):** 20
- **Feature subset strategy (RF_FEATURE_SUBSET_STRATEGY):** `sqrt`
- **Random seed (RF_SEED):** `RANDOM_SEED` (42)
- **Class imbalance:** calculate class weights from the training split and provide a Spark weight column when the positive class is materially under-represented.
- The scikit-learn `min_samples_split` parameter has no direct Spark ML equivalent and is not separately reproduced.

Computational constraint (Databricks free edition)

The original configuration (300 trees, maximum depth 10, minimum instances per node 5) was reduced to the values above for **computational reasons**, not because the modelling approach changed. Databricks free edition runs notebooks on **Serverless** compute, which imposes hard limits that directly bound how large a fitted Spark ML model may grow:

- **100 MB maximum per model**
- **1 GB total in-memory model cache per session**

A 300-tree ensemble on this feature vector repeatedly approached those limits during Sprint 2 execution: training took several minutes per attempt, `mlflow.spark.log_model` required a Unity Catalog volume staging path, and repeated fits in one session failed with `ML_CACHE_SIZE_OVERFLOW_EXCEPTION` until the compute session was reset.

The approved reduction trades ensemble capacity for a model that fits reliably within the free-tier memory cap while preserving the approved workflow (Spark ML Random Forest, same feature set, same split, same 0.50 threshold, same output schema). Recall remains the primary metric; the lower tree count and depth are expected to change metric values slightly relative to the original configuration but do not alter the Sprint 2 acceptance criteria or the interpretation of limitations L-1 to L-3.

Recorded as amendment **A-5**.

Literature justification for Random Forest (recorded retrospectively)

Timing. Random Forest was selected during Sprint 2 planning and implementation as the single approved binary classifier. The literature justification below was prepared **after** that selection, for documentation and review. It does **not** claim that Kok et al. (2024), Matz et al. (2023), or Sani et al. (2022) were used as the decision basis before coding began, and it does **not** record an in-project comparison of alternative classifiers. Alternative-model benchmarking remains out of scope for Sprint 2; future work may compare Random Forest with baselines such as logistic regression, decision tree, gradient boosting, or XGBoost.

Justification. Random Forest was selected as the binary classification model for the machine-learning component of this project. The model predicts an attrition probability for each synthetic student enrolment record. Records with a predicted probability at or above the approved 0.50 threshold are written to the attrition-risk table, which can then be retrieved by the advisor briefing to support early intervention.

Random Forest is appropriate because the student-aggregate dataset is structured tabular data containing mixed demographic, enrolment, course-related, and academic-progress features. Student attrition risk is unlikely to be explained by one variable only and may depend on interactions between multiple factors such as enrolment load, course level, field of education, academic progress, and student background. Random Forest can model these non-linear relationships more effectively than a single decision tree, while also reducing overfitting by combining many trees.

The cited articles support this modelling choice. Kok et al. (2024) used Random Forest for student dropout prediction and highlighted its suitability due to high prediction accuracy, ability to handle many features, and robustness against missing values and outliers. Matz et al. (2023) used demographic, academic, institutional, and engagement-related features for student retention prediction and found that Random Forest produced higher average AUC than the linear elastic-net model in their main results. Sani et al. (2022) also applied Random Forest to student-attrition binary classification and showed stronger performance than a simpler tree-based model.

Random Forest is also useful for this project because it provides feature-importance outputs. This supports the advisor-briefing use case: the system can identify students with higher predicted attrition risk and help explain which features contribute more to that risk. Therefore, Random Forest is a defensible first model for the current project scope.

Related implementation notes (not part of the retrospective literature selection). Class weights are derived from the training split only because the positive class is rare (~5.51%). Recall is the primary metric. Amendment **A-5** reduced trees/depth/leaf size for Databricks free-edition Serverless memory limits. Amendment **A-6** replaced fitted Spark ML preprocessing estimators with training-derived medians, categorical levels, and stateless transforms to work around Spark Connect fitted-model size overestimation ([SPARK-57521](#) / `MODEL_SIZE_OVERFLOW_EXCEPTION`); the statistical contract remains train-only reuse for validation, test, and inference. Feature scaling is not applied because the approved Random Forest does not require it.

9. Training, MLflow, and evaluation

- Start one MLflow run for the Sprint 2 Random Forest implementation.
- Log the target, feature list, split seed, threshold, model parameters, source/output table names, training timestamp, and the training-derived preprocessing contract.
- Fit `RandomForestClassifier` directly on the prepared training feature vector.
- Evaluate validation predictions and log recall, accuracy, precision, F1, ROC-AUC, and confusion-matrix counts.
- After validation checks pass, evaluate the same fitted Random Forest and training-derived preprocessing contract on the untouched test split and log the same metrics.
- Treat recall as the primary reported metric; do not compare alternative models or competing feature configurations.
- Log the trained Random Forest model as the model artifact, together with the preprocessing contract required to reproduce the feature vector. Formal registry promotion is not required.

10. Inference and prediction output

- Apply the training-derived preprocessing contract and the fitted Random Forest to every valid student row in the prepared inference dataset. Do not refit preprocessing.

- Extract the positive-class probability as `attrition_risk_probability`.
- Calculate `attrition_risk_percentage = attrition_risk_probability * 100`.
- Set `attrition_risk_flag` to true when `attrition_risk_probability` is greater than or equal to 0.50.
- Create one output row per `student_deidentified_hash`.

Required prediction-table columns:

Column	Purpose
<code>student_deidentified_hash</code>	Logical primary key linking the prediction to the student and supporting application retrieval.
<code>attrition_risk_percentage</code>	Predicted student-attrition risk from 0 to 100.
<code>attrition_risk_flag</code>	Boolean flag produced using the 0.50 probability threshold.
<code>prediction_threshold</code>	Stored value 0.50 for traceability.
<code>mlflow_run_id</code>	MLflow run that produced the prediction.
<code>scored_at</code>	Timestamp at which inference completed.

11. Persistence and verification

- Write the result in Delta format to `workspace.student_aggregate.student_attrition_risk_prediction`.
- Use overwrite mode to replace the latest snapshot only after scoring and validation succeed.
- Verify the table is queryable and contains one row per applicable student.
- Verify risk percentages are between 0 and 100, flags are Boolean, and identifiers are non-null and unique.
- Verify the stored MLflow run identifier matches the completed training run.
- Query the table by `student_deidentified_hash` and confirm that the Databricks application can retrieve the correct percentage, flag, threshold, run identifier, and scoring timestamp.

12. Refinement and human approval

- Routine code corrections that preserve approved behaviour may be implemented, but they must be included in the implementation summary.
- Any proposed refinement that changes model behaviour, approved features, model parameters, target, split, threshold, data grain, prediction-table schema, or approved architecture must be presented to the human reviewer before it is accepted.
- Each material refinement proposal must identify the reason, evidence, affected specification or task, expected impact, and required documentation updates.
- Approved material changes must be reflected in the controlling SDD documents before the affected tasks and user stories are marked complete.

13. Leave unchanged / out of scope

- Do not modify the synthetic-data generator merely to exclude model features.
- Do not train a course-attribution model.
- Do not perform alternative-model benchmarking, competing feature-subset experiments, automated feature selection, broad hyperparameter searches, or research-led model comparison.
- Do not implement formal MLflow promotion governance, automated retraining, drift monitoring, or scheduled deployment.
- Do not implement the generative-AI advisor briefing in this plan.

14. Approved amendments

Material refinements accepted by the human reviewer after the plan was first approved. Each records the reason, evidence, and impact.

A-1: Two features added to the approved feature set (approved 2026-08-06)

- **Change:** add `course_admission_load_category` and `commencing_continuing_period` as predictive features, taking the assembled set from 19 to 21.
- **Reason:** the reviewer directed that all usable STAR-schema attributes be applied. Both columns are fully populated, prediction-time valid, and carry information not already present.
- **Evidence:** `course_admission_load_category` holds two values (Full-time, Part-time) across all 973,770 rows and is a distinct concept from `attendance_mode`. `commencing_continuing_period` is drawn independently of `commencing_continuing` and differs from it on 350,106 rows (35.95%), so it is not a duplicate encoding.
- **Impact:** widens the feature vector by roughly four one-hot columns. No change to target, grain, split, threshold, model parameters, or output schema.

A-2: Canonical ownership retained over exhaustive column inclusion (approved 2026-08-06)

- **Change:** field-of-education attributes from the study area A, study area B, and unit dimensions remain excluded; redundant encodings and zero-variance columns remain excluded.
- **Reason:** these duplicate concepts already owned by the course dimension, or add no information.
- **Evidence:** column-by-column profiling of all 12 tables confirmed the only populated non-key columns in those dimensions are the three field-of-education levels. `commencing_continuing_12m` and `is_international_student` agree with their canonical counterparts on 100% of rows; `is_enrolment` and `is_study_load` are single-valued. Profiling also confirmed that 38 further fact columns are entirely NULL, so no additional attribute exists to include.
- **Impact:** none; confirms the assembled feature set is complete coverage of the usable schema.

A-3: Synthetic-data generator key-collision fix (approved 2026-08-06)

- **Change:** in the data-generation notebook, the synthetic key width was raised from 4 to 12 characters, with a width guard and a post-write dimension-key uniqueness assertion added. The reviewer elected to run the fix and regenerate all Delta tables immediately, superseding the initial decision to defer regeneration until after Sprint 2.

- **Reason:** Spark's `lpad` truncates inputs longer than the requested width, so every dimension index above 9,999 collapsed onto an existing key. Course offering, unit, and unit offering each held only 9,999 distinct keys.
- **Evidence:** joining unit would have inflated the fact table from 973,770 to 7,508,854 rows, and course offering to 8,167,683.
- **Impact:** no change to the approved feature set, target, grain, split, threshold, model parameters, or output schema. The affected dimensions contribute only field-of-education columns already owned by the course dimension, so the fix unlocks no additional features. Section 13's instruction not to modify the generator refers to excluding model features; correcting a key-integrity defect is not covered by it.
- **Regeneration expectations:** `student_deidentified_hash` and `enrolment_deidentified_hash` derive from the row id rather than the padded keys, so student identity is stable. Feature values and target labels are expected to reproduce exactly because `dbldatagen` applies a predefined random seed by default. Only the synthetic key strings and their hashes change. Phase 1 validation must be re-run against the regenerated tables to confirm this before Phase 2 begins.
- **Closed 2026-08-06:** the reviewer confirmed the notebook runs top to bottom against the regenerated tables with every assertion passing, discharging the re-run requirement.

A-4: Split assigned by hashing the student identifier (approved 2026-08-06)

- **Change:** assign the 70/15/15 split with `pmod(hash(student_deidentified_hash, 42), 100)` into buckets 0-69 train, 70-84 validation, 85-99 test, rather than calling `randomSplit([0.7, 0.15, 0.15], seed=42)`.
- **Reason:** `randomSplit` samples per partition, so its result is stable only while the physical plan and partitioning are stable. Spark's documented mitigation is to cache or save the DataFrame before splitting, which serverless compute forbids. Because the modelling DataFrame is recomputed on every action, a replan between the fit action and an evaluation action could silently move rows between splits and leak test data into training.
- **Evidence:** the serverless failure is reproducible — `cache()` raises `NOT_SUPPORTED_WITH_SERVERLESS: PERSIST TABLE is not supported`. Hashing is a pure function of the data, so notebook section 14.2 proves rather than assumes that no student appears in more than one split.
- **Impact:** ratios, seed, determinism, and disjointness are unchanged. No change to the target, grain, features, model parameters, threshold, or output schema. Split membership differs from a `randomSplit` run, so metrics differ slightly.
- **Affected artefacts:** section 6 of this plan, tasks T9 and T10.

A-5: Random Forest parameters reduced for Databricks free-edition memory limits (approved 2026-08-07)

- **Change:** reduce the approved Random Forest configuration from 300 trees / maximum depth 10 / minimum instances per node 5 to **100 trees / maximum depth 7 / minimum instances per node 20**. `RF_FEATURE_SUBSET_STRATEGY` remains `sqrt` and `RF_SEED` remains `RANDOM_SEED (42)`.
- **Reason:** Databricks free edition Serverless compute caps a single fitted Spark ML model at 100 MB and the total in-memory model cache per session at 1 GB. The original configuration repeatedly approached those limits during Sprint 2 execution, causing long training times, session-cache overflow on re-runs, and dependency on a Unity Catalog volume for `mlflow.spark` artifact staging. The reduction keeps the approved Spark ML Random Forest workflow within those platform constraints.

- **Evidence:** reproducible `ML_CACHE_SIZE_OVERFLOW_EXCEPTION` after two or more successful fits in the same Serverless session; training durations of several minutes per attempt on the full training split; Databricks Serverless documentation records the 100 MB per-model and 1 GB per-session caps.
- **Impact:** no change to the target, grain, features, split method, threshold, output schema, or inference population. Metric values may differ slightly from the original configuration. Tasks T15 and T18 must be read against the amended values in section 8.

A-6: Stateless preprocessing and direct Random Forest fitting

- **Change:** work around a Spark Connect fitted-model size bug by refactoring preprocessing and training so that fitted Spark ML preprocessing estimators are no longer part of the trained-object graph, while preserving the approved Random Forest configuration from amendment **A-5** (100 trees / maximum depth 7 / minimum instances per node 20 / `sqrt` / seed 42).

Concretely:

- Removed the fitted Spark ML preprocessing pipeline containing `Imputer`, `StringIndexer`, and `OneHotEncoder` stages.
- Replaced library-fitted preprocessing models with stateless Spark `DataFrame` transformations: numeric medians are calculated from the training split only; categorical domains are learned from the training split only; categorical values are manually converted to deterministic one-hot columns; explicit unseen/NULL category buckets are retained.
- `VectorAssembler` is used only as a `Transformer`, not as part of a fitted `PipelineModel`.
- `RandomForestClassifier` is fitted directly against the prepared feature vector, so the fitted ML object is only the `RandomForestClassificationModel`.
- The same training-derived preprocessing contract is reused for validation, test, and Phase 3 inference, so preprocessing is not leaked or refitted on held-out or scoring data.
- Preprocessing metadata is logged to MLflow (`preprocessing_contract.json`) so numeric medians, categorical levels, vector-column mappings, and assembler inputs remain reproducible.
- Feature-importance handling aggregates manually generated one-hot slots back to the original approved features without relying on a fitted `OneHotEncoderModel`.

Phase 2 / Phase 3 model assembly, scoring, documentation, and session-cleanup logic were updated for the new architecture.

Reason: Spark Connect can incorrectly overestimate fitted ML model size by traversing model parent references into `SparkSession` / `SparkContext` state. That produced `MODEL_SIZE_OVERFLOW_EXCEPTION` failures even after Random Forest complexity had already been reduced under A-5. Moving preprocessing outside fitted Spark ML estimator models shrinks the affected fitted-object graph while retaining the approved modelling requirements. Tracked upstream as Apache Spark issue [SPARK-57521](#).

Evidence: reproducible Serverless / Spark Connect training failures attributable to fitted preprocessing-plus-classifier model graphs; the failure mode persisted after the A-5 parameter reduction; removing fitted `Imputer` / `StringIndexer` / `OneHotEncoder` / `Pipeline` objects left only the Random Forest classification model in the fitted ML graph.

Impact: no change to the target, approved 21-feature source set, split ratios or method, class weighting, **A-5 Random Forest configuration**, evaluation requirements, 0.50 threshold, inference population, or

output schema. A-6 does **not** restore the pre-A-5 parameters (300 / 10 / 5); those remain superseded by A-5. Validation, test, and inference reuse the training-derived preprocessing contract rather than fitting or refitting preprocessing estimators.

Affected artefacts: section 2, section 7, section 9, and section 10 of this plan; tasks T11, T12, T13, T16, T17, T18, and T23; Phase 2 and Phase 3 notebook sections for preprocessing, training, scoring, MLflow logging, and cleanup.

15. Phase 1 review outcome

Phase 1 was approved by the human reviewer on 2026-08-06 on the evidence of a complete top-to-bottom notebook run against the regenerated Delta tables. Tasks T1-T8 are marked complete in `tasks.md`.

Routine corrections reported at the Phase 1 gate

- `cache()` was removed from four places in the notebook. Serverless compute rejects `cache()` and `persist()` with `NOT_SUPPORTED_WITH_SERVERLESS: PERSIST TABLE is not supported`. Where caching would have amortised several counts over one DataFrame, those counts were folded into a single aggregate, so each check now costs one pass rather than one pass per figure. Approved behaviour is unchanged; the same figures, assertions, and messages are produced.
- This constraint carries into every later phase. No stage may cache or persist a DataFrame. Where a stage genuinely needs a reusable materialisation, it must write a Delta table instead.

16. Known limitations recorded at the Phase 1 gate (T31)

Properties of the synthetic dataset that constrain what any model trained on it can achieve. None is a defect in this workflow, and none changes approved behaviour, but each affects how Phase 2 and Phase 3 results must be read.

L-1: Field of education is decoupled from the value that generated the target

The attrition flags are drawn from Beta parameters selected by each fact row's own `broad_field_of_education`. That column is never written to the fact table. The field of education the model can see comes from the course dimension, whose attributes are seeded from a *different* fact row, because a fact row with id `i` is assigned course index `pmod(i, 1544) + 1`. The two coincide only for the first 1,544 rows, roughly 0.16% of the table.

Measured consequence: course, study area A, and study area B field of education agree with one another on about 13.16% of rows, against a chance-agreement rate of 12.99% computed from the observed category frequencies. The three columns are statistically independent. This independently confirms amendment A-2 — study area contributes unrelated labels rather than a second view of the concept.

L-2: All 21 approved features are close to flat against the target

Against a base rate of 5.51%, the widest attrition spread across any single feature is 1.16 percentage points, on `detailed_primary_field_of_education`, where 45 categories of roughly 20,000 rows each would span about 0.8 points by chance alone. Only `eftsl` (4.79% to 5.54% across three large categories) is clearly

beyond sampling noise, and only because EFTSL is drawn conditioned on the true field of education and so acts as a weak proxy for it.

The theoretical ceiling is quantifiable. Because student attrition is the conjunction of two independent draws against the same probability p , the course-attrition rate gives $E[p]$ and the student rate gives $E[p^2]$:

Quantity	Value
Course attrition rate = $E[p]$	23.211%
Student attrition rate = $E[p^2]$	5.514%
$\text{Var}(p)$	0.001265
$\text{SD}(p)$	0.0356

Each student's true attrition probability is p^2 , with mean 0.055 and standard deviation about 0.017. Expect ROC-AUC near 0.5 and precision near the base rate. Low discrimination is the correct result for this data and must not be treated as a reason to alter the approved features, parameters, or threshold.

L-3: Weighted probabilities are not calibrated risk percentages

Section 8 applies class weights, which recentres predicted probabilities near 0.50 and makes the approved 0.50 threshold operate at the decision boundary as intended. The side effect is that the output ceases to be a calibrated probability of attrition: a student scored at 52 does not have a 52% chance of leaving, when the true probability is nearer 5.5%. `attrition_risk_percentage` is therefore a relative ranking score, not a likelihood, and the Phase 3 output and any downstream application must describe it that way.

17. Phase 2 review outcome

Phase 2 was approved by the human reviewer on 2026-08-06 and tasks T9 to T21 are marked complete in `tasks.md`. Material refinement M-1 was approved and is recorded above as amendment A-4. The following routine corrections were reported at the gate and change no approved behaviour:

- Recall, precision, and F1 are reported for the positive class rather than as weighted averages. A weighted average over a 94.5% negative class is dominated by the majority and would misstate the primary metric.
- The predicted class is derived by applying the 0.50 threshold to the positive probability rather than read from the classifier's own `prediction` column, so the reported metrics are identical to those the Phase 3 risk flag reproduces.
- One-hot importance slots are summed back to their approved feature. Reporting only; no feature is added, removed, or reweighted.
- The MLflow registry URI is pinned to a local path and autologging is disabled. Registry promotion is out of scope for Sprint 2, and the run therefore holds exactly the approved configuration.

One further routine correction was reported after the gate, during first execution:

- Unity Catalog volume for `mlflow.spark` artifact staging.** A Spark ML model is a directory written through Hadoop rather than a driver-local file, and serverless compute cannot see local disk. `mlflow.spark.log_model` therefore fails with UC volume path must be provided to save, log or

load SparkML models in Databricks shared or serverless clusters. The notebook now creates `workspace.student_aggregate.mlflow_tmp` idempotently and passes it as `dfs_tmpdir`. The volume is a staging path only: the model artifact still lands in the MLflow run, and no modelling data is written to it. A write-and-delete smoke test runs before training, because the fit takes several minutes and this failure would otherwise surface only after paying for it. No approved behaviour changes.

- **Serverless model-cache handling.** Serverless caps a single model at 100 MB and all in-memory models in a session at 1 GB, and Spark Connect keeps a fitted model on the driver while any Python reference survives, including references held by the stored traceback of a cell that raised. Repeated fits in one session therefore fail with `ML_CACHE_SIZE_OVERFLOW_EXCEPTION`. The notebook now releases the previous fit before training. No approved behaviour changes.
- **Fitted tree-count assertion.** Serverless stops tree training early when a model approaches the 100 MB per-model cap, which would silently produce fewer trees than the approved ensemble size. The fitted ensemble size is now asserted against `RF_PARAMETERS` immediately after the fit, so a platform truncation is caught rather than being mistaken for the approved configuration.

Phase 2 approval also authorised the Phase 3 Delta write. Persistence remains conditional on every pre-write validation in T27 passing.

18. Phase 3 implementation notes

Recorded at the Phase 3 review gate and approved on 2026-08-06. No material refinement was proposed.

- **Inference population.** Every valid student row in the Phase 1 canonical modelling projection is scored, per section 10 and clarified decision 1. This includes rows used in training and rows whose outcome is unknown. Training rows are scored optimistically, which affects stored percentages but not the Phase 2 metrics, which were measured on the untouched test split. Rows with an unknown outcome could not be trained on but are exactly the students the application needs a prediction for.
- **Threshold application.** The flag is derived from the unrounded probability, so a probability of exactly 0.50 is true and 0.49 is false. A validation check compares the stored flag against the unrounded rule row by row, which proves the flag was never derived from the rounded percentage.
- **Persisted schema.** Exactly the six approved columns. The unrounded probability is computed and validated as an intermediate but deliberately not persisted, because widening the stored schema would need approval and an amendment here. It is carried in the transient staging table described below, which is dropped before the notebook finishes, so no additional table survives the run.
- **Validation ordering.** All checks in T27 run and must pass before anything is written to `student_attrition_risk_prediction`; the publish cell re-asserts the result so it cannot be run out of order. Post-publish checks query the approved table itself, so they verify what the application will read rather than what was intended.
- **Serverless constraint and the scoring staging table.** Consistent with section 15, nothing is cached. Because the modelling DataFrame is a lazy chain of eleven joins and the classifier is a 100-tree forest, validating a lazy scored DataFrame and then writing it would apply the whole chain and the whole forest to the full population twice. Section 15 already prescribes the mitigation: write Delta where a stage needs a reusable materialisation.

The scored result is therefore materialised once into a transient staging table,

`student_attrition_risk_prediction__scoring_staging`. All T27 validation runs against that table as columnar Delta scans. The approved table is replaced only after every check has passed, projected down to the six approved columns, and the staging table is dropped on both the success and the failure path.

This preserves the T27 guarantee exactly — nothing reaches the approved table until validation has passed— and strengthens it, because validation now inspects bytes that survived a round trip through Delta rather than a plan that would be recomputed. It also lets the unrounded probability be validated without appearing in the approved schema, since staging can carry a column the published table does not.

Net effect: the eleven-join chain and the forest are applied to the full population once. The population size is taken from the Phase 1 assertions in section 10 rather than recounted, the distribution and sample read the published table, and the retrieval test compares the published rows against expectations captured from staging before publication. A naive ordering would have cost three full passes.

19. Phase 3 review outcome

Phase 3 was approved by the human reviewer on 2026-08-06 and tasks T22 to T32 are marked complete in `tasks.md`. No material refinement was proposed in Phase 3. The following routine corrections were reported at the gate and change no approved behaviour:

- **Transient scoring staging table.** The scored result is materialised once to `student_attrition_risk_prediction__scoring_staging`, validated there, published to the approved table, and the staging table is dropped. This applies the eleven-join chain and the 100-tree forest to the full population once rather than twice, while preserving the T27 guarantee that nothing reaches the approved table until every check has passed.
- **Unrounded probability validated but not persisted.** Staging carries `attrition_risk_probability` so the flag can be proved to have been derived from the unrounded probability rather than from the rounded percentage. The approved persisted schema remains the six columns in section 10.
- **Inference population carried from Phase 1.** The row count, uniqueness, and outcome breakdown are taken from the Phase 1 section 10 assertions rather than recounted, and the output row count is checked against that figure in T27.
- **Post-publish verification reads Delta.** Distribution, sample, and retrieval tests query the published table rather than rescore. Retrieval expectations are captured from staging before publication so the staging-to-approved copy can be proved exact.

The approved output table `workspace.student_aggregate.student_attrition_risk_prediction` was confirmed queryable with one row per `student_deidentified_hash`. Limitations L-1 to L-3 continue to govern how the stored percentages and flags should be read by the Databricks application.

Sprint 2 machine-learning implementation is complete. Parent Product Backlog stories US-05, US-06, and US-07 remain the project-level acceptance gate; this Sprint 2 specification set is closed.

20. References cited in the Random Forest literature justification

- Kok, C. L., Ho, C. K., Chen, L., Koh, Y. Y., & Tian, B. (2024). A novel predictive modeling for student attrition utilizing machine learning and sustainable big data analytics. *Applied Sciences*, 14(21), 9633. <https://doi.org/10.3390/app14219633>
- Matz, S. C., Bukow, C. S., Peters, H., Deacons, C., Dinu, A., & Stachl, C. (2023). Using machine learning to predict student retention from socio-demographic characteristics and app-based engagement metrics. *Scientific Reports*, 13, 5705. <https://doi.org/10.1038/s41598-023-32484-w>
- Sani, G., Oladipo, F. O., Ogbuju, E., & Agbo, F. J. (2022). Development of a predictive model of student attrition rate. *Journal of Applied Artificial Intelligence*, 3(2), 1–12. <https://doi.org/10.48185/jaai.v3i2.601>

Tasks: Student Attrition Machine Learning

Atomic, testable, dependency-ordered Sprint 2 tasks derived from the specification and plan.

Legend: [] Planned, [x] Completed. Each task maps to one or more independently numbered machine-learning specification stories.

Milestone 1: Modelling-data preparation

Completed and approved at the Phase 1 human-review checkpoint on 2026-08-06. The reviewer confirmed that `student_attrition_machine_learning.py` runs top to bottom against the regenerated Delta tables with every validation assertion passing.

- [x] **T1.** Define source-table, output-table, target, identifier, threshold, and seed constants.
Satisfies: US-1, US-2, US-3, US-4.
- [x] **T2.** Read the synthetic fact table and required populated dimensions from `workspace.student_aggregate`.
Satisfies: US-1, US-5.
- [x] **T3.** Join required descriptive attributes and produce one candidate modelling row per `student_deidentified_hash`.
Satisfies: US-5.
- [x] **T4.** Validate required columns, data types, non-null identifiers, and binary target values.
Satisfies: US-5.
- [x] **T5.** Check row counts and confirm joins have not produced duplicate student identifiers.
Satisfies: US-5.
- [x] **T6.** Report target class distribution and feature missingness; remove unusable all-null features.
Satisfies: US-5.
- [x] **T7.** Define `is_twelve_month_student_attrition` as the label and separate identifiers from predictive features.
Satisfies: US-4.
- [x] **T8.** Explicitly exclude `is_twelve_month_course_attrition`, the target, and identifier hashes from the feature vector.
Satisfies: US-4.

Milestone 2: Split and preprocessing

Completed and approved at the Phase 2 human-review checkpoint on 2026-08-06. Material refinement **M-1**— split assigned by hashing the student identifier rather than `randomSplit`, because serverless compute forbids the `cache()` that `randomSplit` needs to be reproducible — was approved and is recorded as amendment A-4 in `plan.md`.

- [x] **T9.** Create deterministic 70/15/15 training, validation, and test splits using seed 42.
Satisfies: US-6.
- [x] **T10.** Record split row counts and class distributions and verify no row overlaps across splits.
Satisfies: US-6.

- [x] **T11.** Implement numeric casting and training-derived median imputation.
Satisfies: US-7.
- [x] **T12.** Derive categorical levels from training data and implement deterministic manual one-hot encoding with explicit unseen/NULL handling.
Satisfies: US-7.
- [x] **T13.** Assemble processed columns into the Spark ML feature vector and verify excluded columns are absent.
Satisfies: US-4, US-7.

Milestone 3: Random Forest training and evaluation

Completed and approved at the Phase 2 human-review checkpoint on 2026-08-06. T19 to T21 must be read against limitations L-1 to L-3 in `plan.md`: ROC-AUC near 0.5 and precision near the 5.51% base rate are the expected result for this synthetic dataset, and are attributable to the data generating process rather than to the model.

- [x] **T14.** Calculate a training weight column when class imbalance requires it.
Satisfies: US-1, US-8.
- [x] **T15.** Configure the Spark ML Random Forest with 100 trees, maximum depth 7, minimum instances per node 20, `sqrt` feature strategy, and seed 42 (amendment A-5; reduced for Databricks free-edition Serverless memory limits).
Satisfies: US-1.
- [x] **T16.** Implement the stateless preprocessing workflow and configure VectorAssembler as a Transformer for the Random Forest feature vector.
Satisfies: US-1, US-7.
- [x] **T17.** Start an MLflow run and log the approved configuration, feature list, target, split details, threshold, table names, and the training-derived preprocessing contract.
Satisfies: US-1.
- [x] **T18.** Fit RandomForestClassifier directly on the prepared training split and log the trained model artifact and preprocessing contract.
Satisfies: US-1.
- [x] **T19.** Evaluate validation predictions and log recall, accuracy, precision, F1, ROC-AUC, and confusion-matrix counts.
Satisfies: US-8.
- [x] **T20.** Confirm probabilities and predicted classes are non-trivial and investigate any constant-output failure.
Satisfies: US-8.
- [x] **T21.** Perform final evaluation on the untouched test split and log the same metrics.
Satisfies: US-6, US-8.

Milestone 4: Inference and Delta persistence

Completed and approved at the Phase 3 human-review checkpoint on 2026-08-06. The reviewer confirmed that `workspace.student_aggregate.student_attrition_risk_prediction` was written with one row per `student_deidentified_hash`, that retrieval by the logical primary key returns the correct result, and that no source table was modified. T23, T27, and T28 are implemented through a transient scoring staging table so the model is applied to the full population once rather than twice; the guarantee T27 requires — nothing

persisted to the approved table until every check passes — is unchanged. See sections 18 and 19 of `plan.md`.

- [x] **T22.** Build the applicable-student inference dataset using the same validated column definitions.
Satisfies: US-2, US-5.
- [x] **T23.** Apply the training-derived preprocessing contract and fitted Random Forest to every valid applicable student row.
Satisfies: US-2.
- [x] **T24.** Extract the positive-class probability and calculate `attrition_risk_percentage` from 0 to 100.
Satisfies: US-2.
- [x] **T25.** Apply the fixed 0.50 probability threshold to produce a Boolean `attrition_risk_flag`.
Satisfies: US-2.
- [x] **T26.** Create the prediction output with `student_deidentified_hash`, `percentage`, `flag`, `threshold`, MLflow run ID, and scored timestamp.
Satisfies: US-2, US-3.
- [x] **T27.** Validate non-null unique student identifiers, valid percentage range, Boolean flags, and one output row per applicable student.
Satisfies: US-3.
- [x] **T28.** Overwrite `workspace.student_aggregate.student_attrition_risk_prediction` with the validated latest snapshot.
Satisfies: US-3.
- [x] **T29.** Query the written Delta table and confirm the Databricks application can retrieve the correct prediction result by `student_deidentified_hash`.
Satisfies: US-3.

Milestone 5: Sprint 2 review

Completed at the Phase 3 human-review checkpoint on 2026-08-06. The reviewer approved the implementation against the controlling documents, accepted the recorded limitations L-1 to L-3, and authorised closure of Sprint 2 machine-learning tasks.

- [x] **T30.** Review AI-generated code, reported technical corrections, and proposed refinements against the constitution, specification, plan, tasks, and acceptance criteria. Obtain human approval for every material refinement before accepting it.
Satisfies: All stories.
- [x] **T31.** Record known limitations, including synthetic-data assumptions and the absence of real-data validation.
Satisfies: US-8. Recorded as L-1 to L-3 in `plan.md`.
- [x] **T32.** Mark tasks and user stories complete only after successful execution, testing, and human review.
Satisfies: All stories.